

Dossier Technique Projet Bowl

BDD5 2025/2026

DUTHY Aurélien

CHARIERE Lilian

FELTIN Quentin

DEBAISIEUX Corentin

SOMMAIRE

1. INTRODUCTION

- 1.1 Contexte du projet
- 1.2 Objectifs de la plateforme "Bowl"
- 1.3 Public visé

2. ARCHITECTURE PROJET

- 2.1 Base de données (MySQL)
- 2.2 Structure de l'arborescence des fichiers
- 2.3 Diagramme de fonctionnement global

3. Installation de la plateforme

4. DÉVELOPPEMENT BACK-END

- 4.1 Gestion de l'authentification et sessions

5. DÉVELOPPEMENT FRONT-END

- 5.1 Récupération dynamique des données
- 5.2 Intégration de la carte interactive

6. BILAN DU PROJET

- 7.1 État actuel du projet
- 7.2 Améliorations futures possibles

Introduction :

Le projet confié à notre groupe consiste à concevoir une plateforme web dédiée à la gestion des stages et des alternances proposés par l'IUT. Cette plateforme remplacera l'ancienne qui à ce jour n'est pas pratique et pas maintenable à cause de son code, elle a pour objectif de centraliser les échanges entre les différents acteurs :

- **Pour les enseignants :** Elle permet d'ajouter de nouvelles offres et d'assurer un suivi personnalisé de chaque étudiant en visualisant l'historique de ses candidatures ainsi que l'état d'avancement de ses résultats.
- **Pour les étudiants :** Elle simplifie leurs démarches en offrant une interface unique pour déposer leur CV et leur lettre de motivation. Ces documents deviennent alors accessibles aux employeurs auprès desquels l'étudiant a postulé.
- **Suivi des candidatures :** La plateforme remplace le suivi manuel (format papier) par un tableau de bord dynamique permettant de savoir en temps réel si une candidature est en attente, refusée ou acceptée.

La plateforme **Bowl** s'adresse principalement aux étudiants et à l'administration de l'IUT Lyon 1. Pour ce faire, nous avons choisi de concevoir une interface reprenant les codes visuels et l'aspect de **Moodle**, un outil déjà largement utilisé à l'Université Lyon 1.

Ce choix a été crucial dès le début du projet : il garantit une simplicité de compréhension immédiate pour tous les utilisateurs et facilite l'adoption de la plateforme grâce à un environnement déjà familier.

Architecture du projet :

2.1 Base de données

Le cœur du système repose sur la liaison entre la base de données et le serveur, garantissant un site dynamique et sécurisé.

1. Base de données MySQL et PDO : Nous utilisons **MySQL** pour stocker les informations (profils, offres). La connexion s'effectue via l'extension **PDO** en PHP, ce qui permet de sécuriser les échanges et de neutraliser les tentatives d'injections SQL grâce à la méthode `$pdo->quote()`.

2. Modularité des requêtes SQL : Pour faciliter la maintenance, les requêtes sont isolées dans des fichiers externes (ex: `getUserInfo.SQL`). Le script PHP récupère ce code, y injecte les variables sécurisées, et exécute la demande auprès de la base de données.

3. Format d'échange JSON : Une fois les données récupérées, le serveur les transforme au format **JSON**. Ce format léger est ensuite envoyé au client (navigateur) pour permettre une mise à jour immédiate de l'affichage sans avoir à recharger la page.

2.2 Structure de l'arborescence des fichiers

Pour une meilleure gestion de l'ensemble du système, nous avons opté pour une organisation du projet en plusieurs sous-dossiers. C'est-à-dire que chaque espace comprenant plusieurs pages possède ses propres sous-répertoires suivant une structure identique : `index.html`, `script.js`, `index.php`, `style.css` et si besoin un fichier `script.SQL`. Ce choix a été fait au lieu de multiplier les fichiers `.html` et autres avec des noms différents au sein d'un même dossier, ce qui aurait rendu la maintenance difficile.

Voici un exemple de structure de dossier de notre projet :

📁 SAEWeb (DOSSIER MERE)

|

| — 📁 Etudiant

| | — 📁 Offres

| | | — `index.html`, `style.css`, `scripts.js`, `offres.php`, `script.SQL`

| | — 📁 Profil

| | | — `index.html`, `style.css`, `scripts.js`, `index.php`, `getUserInfo.SQL`

| | — 📁 Candidatures

```

|   └─ index.html, style.css, scripts.js, index.php
|
|─ └─  Prof
|
|   └─  GestionOffres (Ajout/Modification d'offres)
|     └─ index.html, style.css, scripts.js, index.php, script.SQL
|
|   └─  SuiviEtudiants (Visualisation des parcours)
|     └─ index.html, style.css, scripts.js, index.php
|
|─ └─  Entreprise
|
|   └─  MesOffres
|     └─ index.html, style.css, scripts.js, index.php
|
|   └─  ConsultationCV
|     └─ index.html, style.css, scripts.js, index.php
└─ .....

```

2.3 Diagramme de fonctionnement BDD :

CONTACTS		
int	ContactID	PK
string	ContactLastName	
string	ContactFirstName	
string	ContactGender	
string	ContactEmail	

3. Installation plateforme :

Mise a part les codes sources html github Il n'y a aucune installation viable de plateforme étant donné que nous n'avons pas pu répondre aux attentes dans les temps impartis notamment celle-ci qui était de créer une plateforme interactive.

4. DÉVELOPPEMENT BACK-END

4.1 Gestion de l'authentification et sessions (CAS)

Plutôt que de développer un système d'authentification interne, nous avons fait le choix stratégique d'utiliser le protocole **CAS (Central Authentication Service)**.

En déléguant l'authentification à l'infrastructure de l'Université Lyon 1, nous évitons de stocker des mots de passe dans notre base de données. Cela élimine les risques critiques liés aux fuites de données et aux attaques par force brute sur notre serveur.

Ce système permet une connexion unique. Puisque les étudiants et professeurs sont déjà connectés à leur mail ou à Moodle, l'accès à **Bowl** se fait de manière fluide sans qu'ils aient à mémoriser un nouvel identifiant.

Implémentation Technique :

- **Vérification de Session** : Au début de chaque fichier PHP, nous vérifions si l'utilisateur possède une session active.
- **Redirection Conditionnelle** : Si la variable de session `isLoggedIn` n'est pas détectée, le script redirige automatiquement l'utilisateur vers l'URL du service CAS.
- **Token de Session** : Une fois authentifié, un `sessionToken` unique est attribué à l'utilisateur. Ce jeton est ensuite utilisé comme clé sécurisée pour interroger la base de données et récupérer les informations spécifiques au profil sans jamais exposer l'identifiant réel dans les requêtes URL.

5.1 Récupération dynamique des données (Méthodes GET et POST)

L'interactivité de la plateforme **Bowl** repose sur l'échange de données entre le client et le serveur via les méthodes de transfert HTTP standard (**GET** et **POST**).

1. Utilisation de la méthode GET : Pour la simple consultation d'informations (comme l'affichage de la liste des stages), nous utilisons principalement la méthode **GET**.

- **Fonctionnement :** Le script JavaScript envoie une requête au serveur PHP pour "récupérer" les ressources. Cette méthode est privilégiée ici car elle est légère et permet une lecture rapide des données stockées dans MySQL sans modification de la base.

2. Utilisation de la méthode POST pour la sécurité : Pour les actions sensibles, comme l'envoi d'un token d'authentification ou le dépôt d'une candidature, nous privilégions la méthode **POST**.

- **Avantage technique :** Contrairement au GET, les données envoyées en POST ne sont pas visibles dans l'URL du navigateur. Cela renforce la sécurité des échanges et permet de transmettre des volumes de données plus importants (comme le contenu d'un CV ou d'une lettre de motivation) sans restriction de taille.

3. Traitement et formatage JSON : Quel que soit le mode de transfert choisi (GET ou POST), le serveur PHP traite la demande et retourne une réponse structurée au format **JSON**. Ce format est idéal car il est facilement interprété par le JavaScript pour extraire les informations nécessaires à l'affichage.

4. Injection dynamique dans le DOM : Une fois les données reçues via ces protocoles, nous utilisons des fonctions JavaScript pour manipuler le **DOM** (Document Object Model). Par exemple, la fonction `afficherOffres(liste)` génère dynamiquement les balises HTML nécessaires. Cela permet de mettre à jour les cartes d'offres ou les informations du profil instantanément, offrant une expérience fluide sans nécessiter de rechargement complet de la page.

5.2 Intégration de la carte interactive (Leaflet.js)

L'une des fonctionnalités majeures de **Bowl** est la visualisation géographique des opportunités de stage via une carte interactive.

1. Choix de la bibliothèque : Nous avons utilisé **Leaflet.js**, une bibliothèque JavaScript "Open Source" légère, pour intégrer des fonds de carte **OpenStreetMap**. Ce choix permet d'éviter les solutions payantes (comme Google Maps) tout en offrant une fluidité optimale sur mobile et ordinateur.

2. Logique de géolocalisation : Le script JavaScript traite les données de la manière suivante

- **Récupération** : Le JS reçoit la liste des offres (contenant le nom de la ville) via la méthode GET.
- **Conversion** : Comme la base de données ne contient pas de coordonnées GPS brutes, nous utilisons MapBOX en réalisant une requête en fournissant l'adresse postale en format textuel et nous renvoie les coordonnées (Latitude/Longitude). **A l'avenir on les stockera sûrement dans notre base de données.**
- **Affichage** : La fonction L.marker() est appelée pour chaque offre afin de placer un marqueur précis sur la carte.

3. Interactivité : Chaque marqueur est lié à une "popup" interactive. En cliquant sur un point, l'étudiant voit apparaître instantanément les détails de l'offre (Entreprise, intitulé du poste) et un lien direct pour postuler, créant ainsi un pont direct entre la cartographie et le contenu textuel.

Bilan du projet :

À ce jour, le projet **Bowl** a atteint un stade de développement estimé à environ **40 %** ce qui est assez déroutant et catastrophique.

1. Réalisations (Front-end) : La phase de conception visuelle et d'interface utilisateur est entièrement finalisée. L'ensemble des pages web, l'aspect graphique (UI) et l'ergonomie (UX) sont opérationnels. Le design responsive et l'intégration de la carte interactive Leaflet sont terminés et validés.

2. Travaux en cours et perspectives (Back-end) : La partie Back-end est actuellement le principal axe de développement. Si l'architecture PHP est en place, plusieurs points cruciaux restent à finaliser :

- **Validation de la base de données** : Bien que les schémas soient définis, des tests approfondis sont nécessaires pour confirmer l'intégrité des relations entre les tables.
- **Débogage des requêtes SQL** : Nous devons encore valider la communication réelle entre le PHP et MySQL pour nous assurer que chaque requête (comme la récupération des profils ou des offres) retourne les résultats attendus sans erreur.
- **Fiabilité du système** : Une phase de tests unitaires est prévue pour vérifier le bon fonctionnement du flux de données complet (du clic sur l'interface jusqu'à la mise à jour en base de données).

Conclusion du bilan : Le projet dispose d'une base front-end solide et d'une structure serveur cohérente. L'étape finale consistera à stabiliser les échanges de données pour transformer cette interface visuelle en un outil pleinement dynamique et sécurisé

Amélioration :

Idée 1 :

Malheureusement cela ne sera valable uniquement pour les offres proposées par l'IUT. Il va de soi que chaque étudiant ne pourra s'appuyer uniquement sur cet outil car il existe une multitude d'offres externes à l'IUT qui doivent être prises en compte. Peut-être que nous pourrions mettre en place un système d'ajout d'entreprises où l'étudiant ajoute à son tableau de bord toutes les offres auxquelles il aurait postulé en dehors de celle de l'IUT

Pour répondre à cette problématique, l'une des évolutions les plus pertinentes pour la version 2.0 de Bowl serait l'intégration d'un système d'ajout manuel d'entreprises et d'offres externes.

Le concept technique :

- **Formulaire de saisie libre :** Nous pourrions mettre en place une interface permettant à l'étudiant de renseigner lui-même les informations d'une candidature effectuée hors plateforme (Nom de l'entreprise, intitulé du poste, lien de l'offre, date de dépôt).
- **Centralisation des statuts :** Ces offres externes seraient listées sur le même tableau de bord que les offres IUT, permettant un suivi unifié des états (« En attente », « Entretien prévu », « Refusé »).
- **Bénéfice pédagogique et administratif :** Cette fonctionnalité transformerait Bowl en un véritable outil de pilotage de carrière. Les professeurs auraient ainsi une vision exhaustive de l'investissement de l'étudiant dans sa recherche, même si celle-ci se déroule en dehors du réseau de l'IUT.

Idée 2 :

Système de notifications et "Push" en temps réel

Pour améliorer le suivi des candidatures, le site pourrait intégrer un système de notifications automatisées.

- **Le concept :** Utiliser des **WebSockets** ou des tâches planifiées (Cron jobs) en PHP. Dès qu'un professeur ou une entreprise change le statut d'une candidature (ex: de "En attente" à "Accepté"), l'étudiant reçoit immédiatement une alerte sur son tableau de bord ou par mail.
- **Bénéfice :** Cela renforcerait l'aspect "temps réel" de la plateforme et réduirait le stress des étudiants qui n'auraient plus besoin de se connecter de manière répétitive pour vérifier l'état de leur dossier.